

Relatório kNN

Eduardo

March 25, 2025

1 Metodologia

O código foi escrito em matlab 2024b.

1.1 Algoritmo de classificação

Foi implementado o algoritmo **k-nearest neighbors** ponderado com *kernel* arbitrário, conforme 1.

Listing 1: Algoritmo kNN

```
function label = mykNN(x, complete_set , k, h)
    % mykNN: binary classifier: label is {-1, 1}
    % x: point to be classified
    % complete_set: training set with class labels
    % k: number of nearest neighbors
    % h: kernel bandwidth
    % label: class label of x

    % number of training samples
    training_set_point_qtty = size(complete_set , 1);

    % number of features
    feature_qtty = size(complete_set , 2) - 1;

    % initialize distances vector
    distances = zeros(training_set_point_qtty , 1);

    % compute distances
    for i = 1:training_set_point_qtty
        distances(i) = euclidean_distance(x, complete_set(i, 1:
            ↪ feature_qtty));
    end

    % sort distances
    [sorted_distances , sorted_indexes] = sort(distances);

    % initialize class labels vector
```

```

class_labels = zeros(k, 1);

% get class labels of k nearest neighbors
for i = 1:k
    class_labels(i) = complete_set(sorted_indexes(i),
    ↪ feature_qtty + 1);
end

% compute kernel weights
weights = zeros(k, 1);

for i = 1:k
    weights(i) = kernel_function(sorted_distances(i), h);
end

% compute class label
label = sign(sum(weights .* class_labels));

end

function distance = euclidean_distance(x, y)
    % euclidean_distance: computes the euclidean distance between
    ↪ two points
    % x, y: points
    % distance: euclidean distance between x and y

    distance = sqrt(sum((x - y) .^ 2));

end

function value = kernel_function(distance, h)
    % kernel_function: computes the kernel function value
    % distance: distance between two points
    % h: kernel bandwidth
    % value: kernel function value

    value = exp(-distance / (2 * h ^ 2));

end

```

1.2 Geração de dados

Para avaliar o classificador, dados são gerados pelas funções em 2.

Listing 2: Geração de dados

```

function [data1, labels1, data2, labels2] = generate_dataset(
    ↪ datasetType, n, noise)
    % Generate synthetic dataset using MATLAB's built-in functions

```

```

% datasetType: Choose between 'blobs', 'spirals', 'moons', '
    ↪ xor', 'circles'
% n: Number of samples
% noise: Noise level

switch datasetType
case 'blobs'
    data1 = mvnrnd([2, 2], [0.5, 0; 0, 0.5], n/2) + noise *
        ↪ randn(n/2, 2);
    data2 = mvnrnd([-2, -2], [0.5, 0; 0, 0.5], n/2) + noise *
        ↪ randn(n/2, 2);
case 'spirals'
    t = linspace(0, 4*pi, n/2)';
    data1 = [t.*cos(t), t.*sin(t)] + noise * randn(n/2, 2);
    data2 = [-t.*cos(t), -t.*sin(t)] + noise * randn(n/2, 2);
case 'moons'
    [data, labels] = make_moons(n, noise);
    data1 = data(labels == 1, :);
    data2 = data(labels == 0, :);
case 'xor'
    data1 = [randn(n/4, 2) + 2; randn(n/4, 2) - 2] + noise *
        ↪ randn(n/2, 2);
    data2 = [randn(n/4, 2) + [-2, 2]; randn(n/4, 2) - [-2, 2]]
        ↪ + noise * randn(n/2, 2);
case 'circles'
    [data, labels] = make_circles(n, noise);
    data1 = data(labels == 1, :);
    data2 = data(labels == 0, :);
otherwise
    error('Unknown dataset type');
end

% Assign labels
labels1 = ones(size(data1, 1), 1);
labels2 = -ones(size(data2, 1), 1);
end

% Helper functions for generating datasets
function [data, labels] = make_moons(n, noise)
    theta = linspace(0, pi, n/2)';
    r = 1 + noise * randn(n/2, 1);
    data1 = [r .* cos(theta), r .* sin(theta)];
    data2 = [1 - r .* cos(theta), -r .* sin(theta) - 0.5];
    data = [data1; data2] + noise * randn(n, 2);
    labels = [ones(n/2, 1); zeros(n/2, 1)];
end

function [data, labels] = make_circles(n, noise)

```

```

theta = linspace(0, 2*pi, n/2)';
r1 = 1 + noise * randn(n/2, 1);
r2 = 0.5 + noise * randn(n/2, 1);
data1 = [r1 .* cos(theta), r1 .* sin(theta)];
data2 = [r2 .* cos(theta), r2 .* sin(theta)];
data = [data1; data2] + noise * randn(n, 2);
labels = [ones(n/2, 1); zeros(n/2, 1)];
end

```

1.3 Cálculo numérico da superfície de decisão

Para permitir a visualização da superfície de decisão, o classificador, treinado, classifica diversos pontos ao longo da região de interesse. 3 produz a malha de pontos.

Listing 3: Geração de malha de pontos

```

function [X, X1, X2] = generate_grid(complete_set, resolution)

% Determine the range of the data
min_x1 = min(complete_set(:, 1));
max_x1 = max(complete_set(:, 1));
min_x2 = min(complete_set(:, 2));
max_x2 = max(complete_set(:, 2));

% Generate a grid of points within the data range
x1 = linspace(min_x1 - 1, max_x1 + 1, resolution);
x2 = linspace(min_x2 - 1, max_x2 + 1, resolution);
[X1, X2] = meshgrid(x1, x2);
X = [X1(:), X2(:), zeros(numel(X1), 1)];

end

```

2 Testes e Resultados

O classificador foi avaliado para diferentes pares de **k** e **r** - a abertura da gaussiana geradora dos pontos de treinamento. Para cada par, a superfície de decisão é calculada, plotada e salva como imagem. 4 mostra o código principal.

Listing 4: Código principal

```

% Generate synthetic dataset using MATLAB's built-in functions
datasetType = 'blobs'; % Choose between 'blobs', 'spirals', '
    ↪ moons', 'xor', 'circles'

% Dataset parameters
samples = 100;
noise = 2;

% kNN parameters

```

```

k = 5;
h = 1;

for k = 1:20:50
    for noise = 1:1:3

        % Generate the dataset
        [data1, labels1, data2, labels2] = generate_dataset(
            ↪ datasetType, samples, noise);
        complete_set = [data1, labels1; data2, labels2];

        % Generate a grid for plotting
        [X, X1, X2] = generate_grid(complete_set, 100);

        % Classify the points
        for i = 1:size(X, 1)
            X(i, 3) = mykNN(X(i, 1:2), complete_set, k, h);
        end

        % Plot the decision boundaries
        plot_boundaries(X, X1, X2, data1, data2, true, string(k) + '
            ↪ _' + string(noise) + '.png');

    end
end

```

As figuras a seguir relacionam as superfícies de decisão aos parametros avaliados.

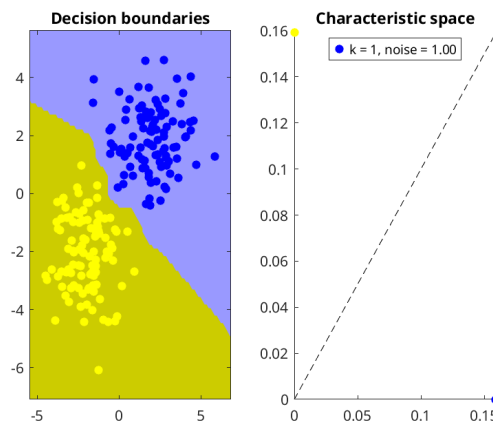


Figure 1: Superfície de decisão e espaço característico para $k=1$ e $r=1$

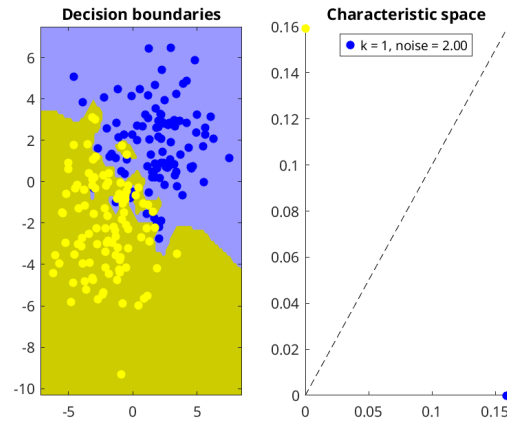


Figure 2: Superfície de decisão e espaço característico para $k=1$ e $r=2$

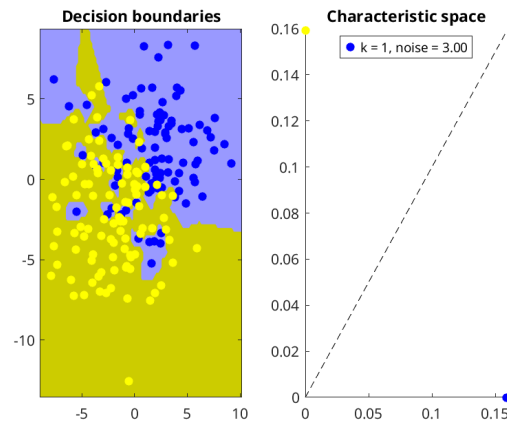


Figure 3: Superfície de decisão e espaço característico para $k=1$ e $r=3$

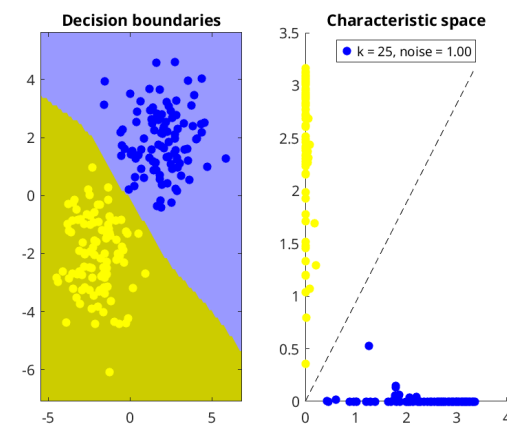


Figure 4: Superfície de decisão e espaço característico para $k=25$ e $r=1$

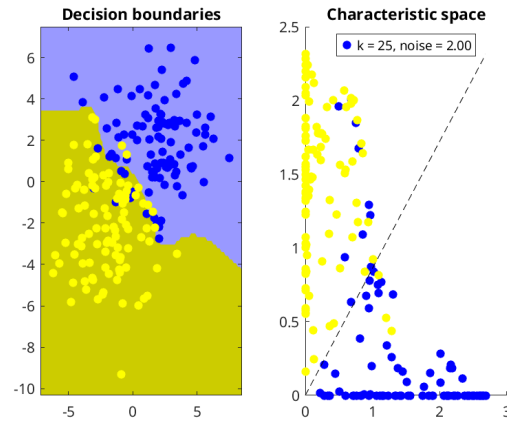


Figure 5: Superfície de decisão e espaço característico para $k=25$ e $r=2$

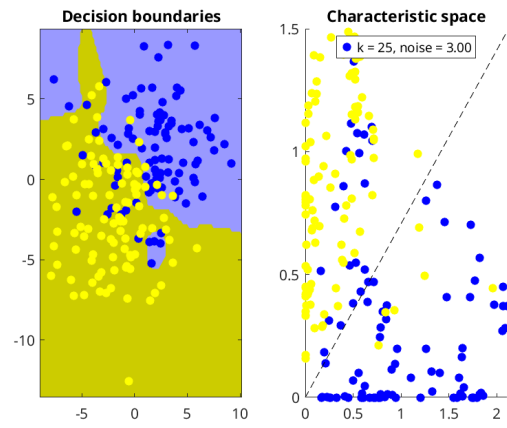


Figure 6: Superfície de decisão e espaço característico para $k=25$ e $r=3$

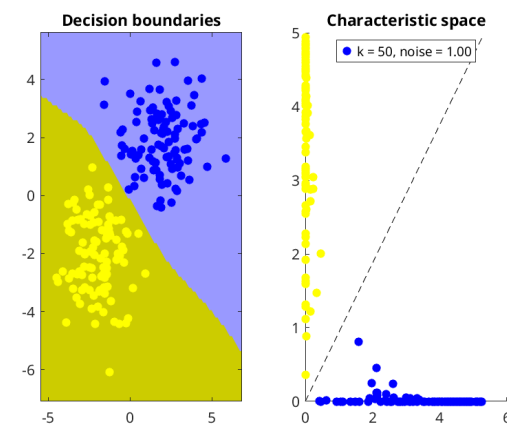


Figure 7: Superfície de decisão e espaço característico para $k=50$ e $r=1$

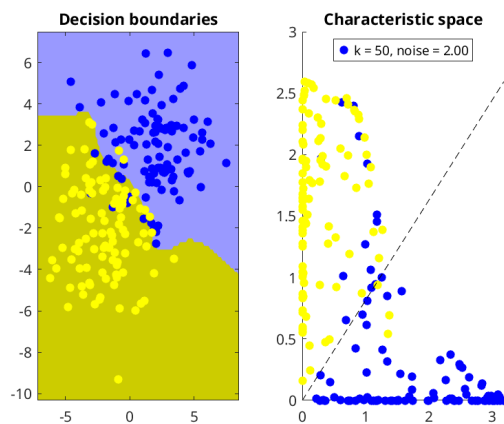


Figure 8: Superfície de decisão e espaço característico para $k=50$ e $r=2$

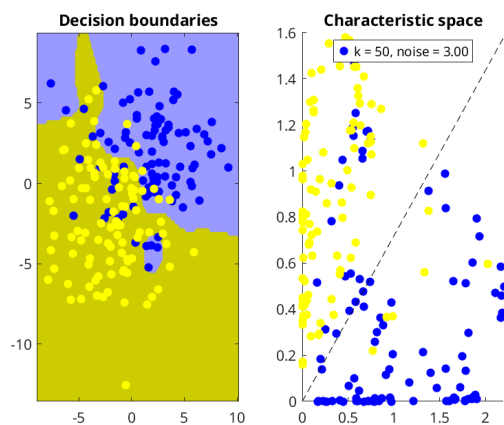


Figure 9: Superfície de decisão e espaço característico para $k=50$ e $r=3$

3 Conclusão

Pelos resultados, fica claro que, quanto maior a quantidade de vizinhos avaliada, maior a robustez do classificador em relação a *outliers*. No entanto, valores muito altos podem fazer com que este não capture com fidelidade a distribuição espacial dos dados.